Computer Programming 1

Introduction to the laboratory
(16-09-2026)

The physical laboratory

- Two operating systems (OS) for each computer
  - Windows and Linux (dual boot)

  → we will use **Linux** … and the **g++ compiler**

- Access via username and password provided by UniTN
  - the same you use to access ESSE3 and the email

Ubuntu

- Operating system (OS) based on Linux

- Distributed under the GNU-GPL license
  - Possibility to use it freely
  - Possibility to freely install proprietary software

https://www.ubuntu.com

A typical Linux distribution comprises:
- a Linux kernel,
- GNU tools and libraries,
- additional software,
- documentation,
- a window system (e.g., X windows System),
- a window manager,
- a desktop environment,
- some proprietary software and drivers.

Other existing Linux distributions are:
- Mint
- Fedora
- openSUSE
- Debian
- Arch
- Mageia
- OpenWrt (embedded devices)
- Rocks Cluster (supercomputers)
- …



… on your PC

Simply have a C++ compiler in your Windows/Mac

To have a local copy of Linux:

1. Add Ubuntu to your PC other than Windows
   - Dual-boot modality
2. Use a **live-USB Linux** from your PC
   - not persistent OS
3. Install a **Virtual Machine** with Ubuntu on the PC
4. Use **Windows Linux Subsystem**

Other solutions exist (no for this course!)

- E.g., use a compiler from browser (e.g., http://cpp.sh, https://repl.it, etc.)

Linux on your PC … *links to start*

Links to start:

Install Ubuntu

- https://ubuntu.com/desktop/docs/en/latest/tutorial/install-ubuntu-desktop

USB-live Linux

- https://www.dell.com/support/kbdoc/en-us/000119771/how-to-create-a-live-ubuntu-linux-usb-key?lwp=rt

Virtual box

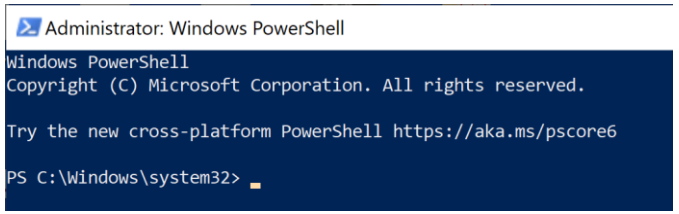
- https://ubuntu.com/tutorials/how-to-run-ubuntu-desktop-on-a-virtual-machine-using-virtualbox#1-overview
- https://itsfoss.com/install-linux-in-virtualbox/

Windows Linux Subsystems

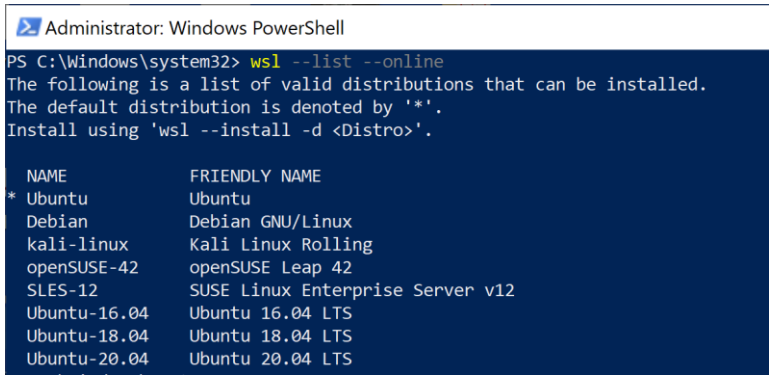
- https://docs.microsoft.com/en-us/windows/wsl/install

Example: WSL - Windows Linux Subsystems

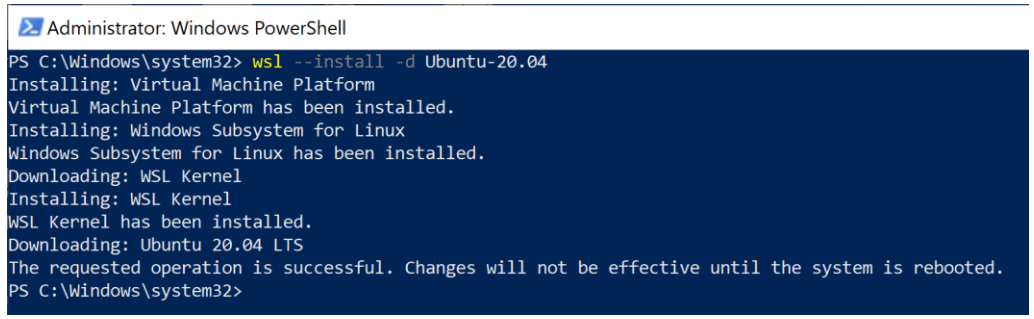
- Environment for running Linux inside Windows



1) Start PowerShell as Administrator

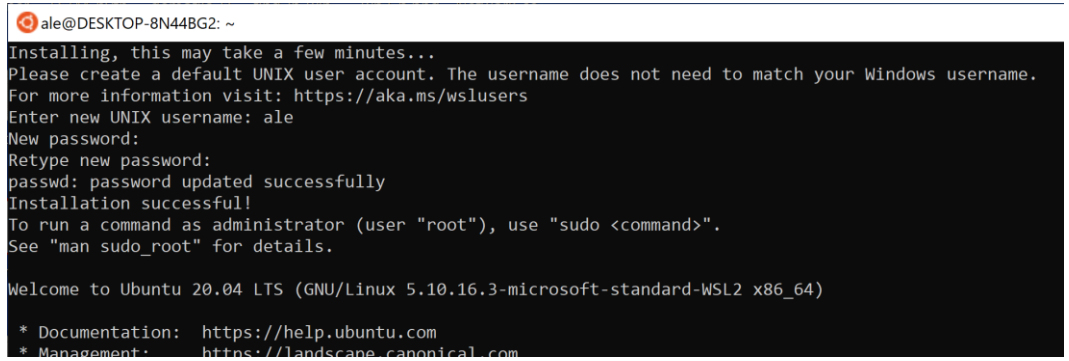


2) Check the available Linux versions
> wsl --list --online



3) Install the latest Ubuntu version
> wsl --install --d Ubuntu-20.04

4) Restart the PC



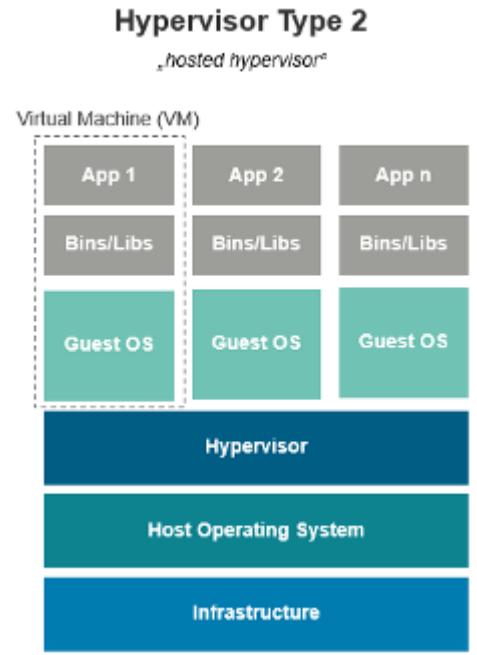
5) Set username + password for the Ubuntu user

Ubuntu ready!

Installation of the compiler:
>sudo apt-get install build-essential

Example: VM installation (1)

- VM basic idea: abstract hardware of a single computer into several different (software) execution environments
  - Layer-based system that creates virtual system (virtual machine - VM) on which operation systems or applications can run
- Main components
  - Host: underlying hardware system
  - Virtual machine manager (VM) or hypervisor: creates and runs virtual machines by providing interface that is (almost) identical to the host
  - Guest: process provided with virtual copy of the host
    - Usually, an operating system (OS)
- Single physical machine can run multiple operating systems concurrently, each in its own virtual machine
- Different types exist (e.g., VM Type 2, containers, etc.)



Example: VM installation (1)

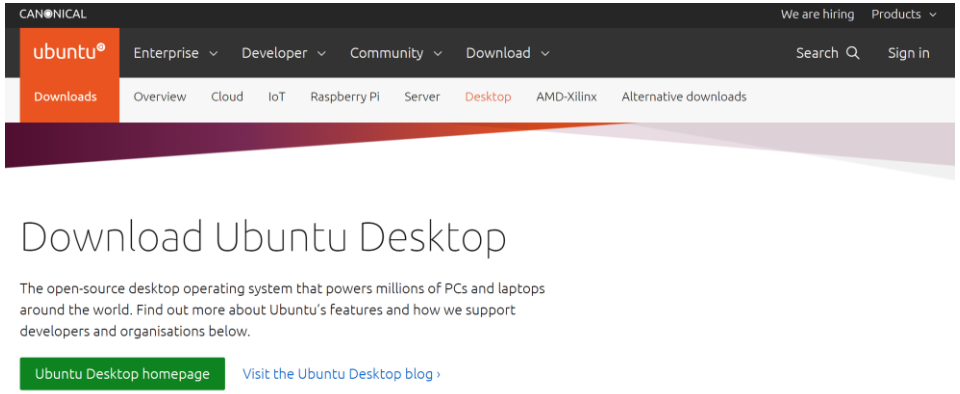
- Step 1: download and install hypervisor
  - VMWare Workstation player (free for students, for non-commercial use):
    - https://www.vmware.com/products/workstation-player/workstation-player-evaluation.html
  - Oracle VirtualBox:
    - https://www.virtualbox.org/





Example: VM installation (2)

- Step 2: Install Ubuntu
  - https://ubuntu.com/download/desktop
  - Current version Ubuntu 22.04 LTS
    - Every version ≥ 18.04 is fine for our purposes
  - Guide on how to install Ubuntu
    - https://ubuntu.com/tutorials/install-ubuntu-desktop#11-installation-complete
  - Help on how to install Ubuntu on top of VirtualBox
    - https://ubuntu.com/tutorials/how-to-run-ubuntu-desktop-on-a-virtual-machine-using-virtualbox#1-overview

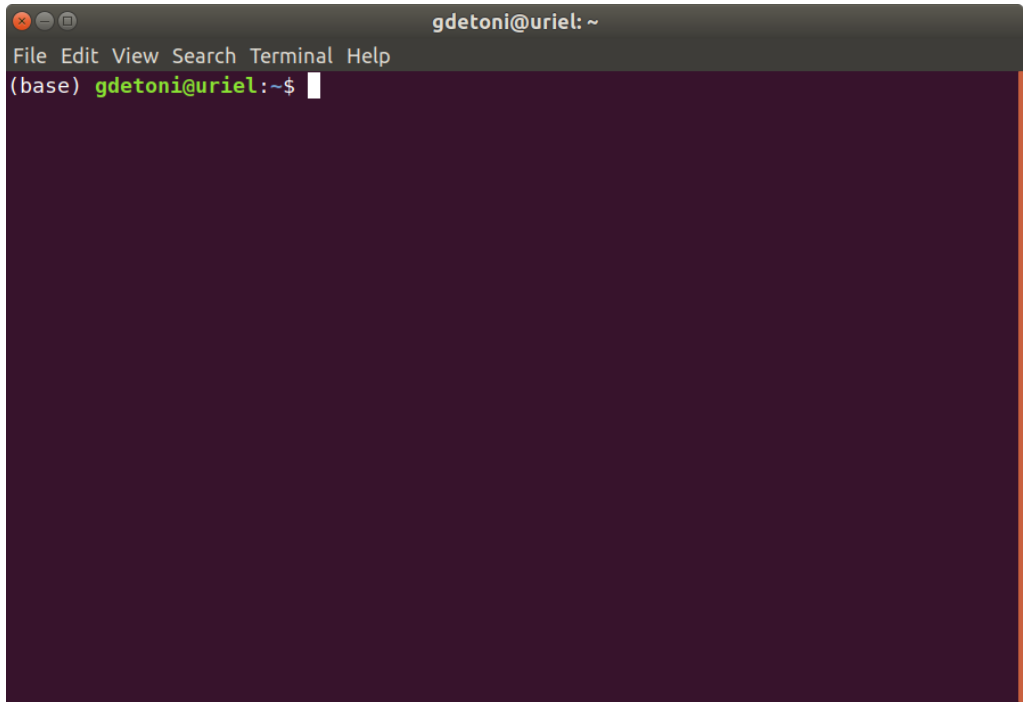


Steps
a) Download and install VirtualBox
b) Download an Ubuntu Image
c) Create a new virtual machine in VirtualBox
d) Install your image (Ubuntu)
e) Changing the screen resolution
f) Installing Guest Additions

Installation of the compiler:
>sudo apt-get install build-essential

Linux SHELL

- Environment for command execution in the Linux OS
- The shell is an interface between the OS and the user
- This interface can be textual (CLI) or graphical (GUI)
  - CLI = Command Line Interface
  - GUI = Graphic User Interface
- All OS have a shell



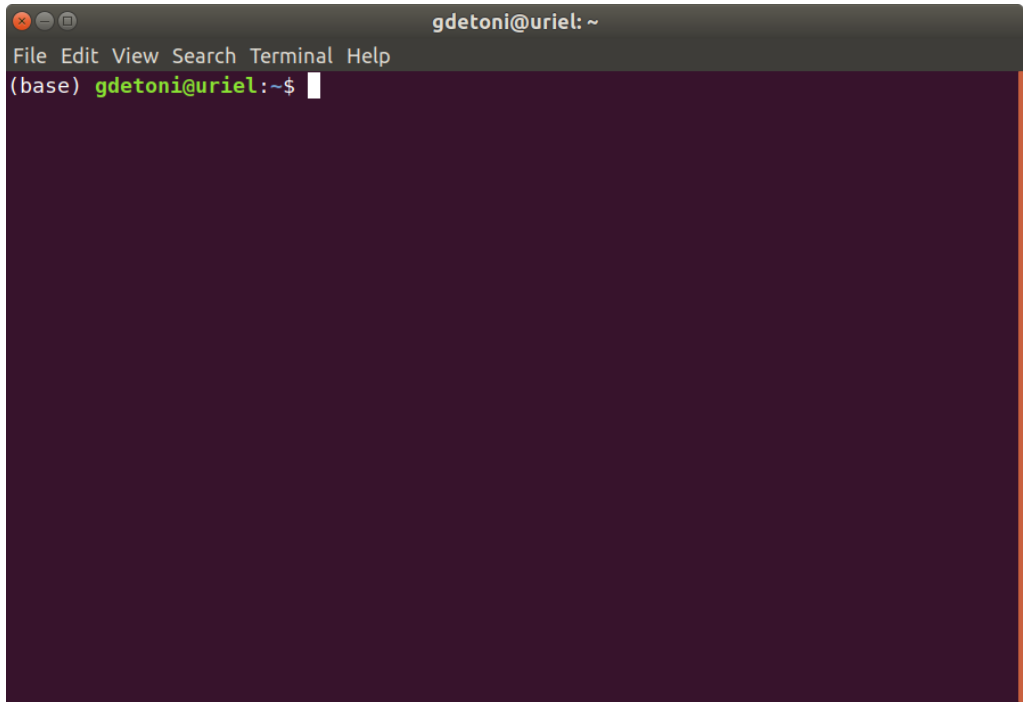
How to access to the SHELL?

- CTRL+ALT+T
- Activity → write: “Terminal”

Linux SHELL (2)

Possible uses

- Execute commands
  - The shell can interpret and execute commands
- Setup and customize the OS
  - The environmental variable of the Linux session can be setup.
  - Default values of such variables can be changed before being loaded at user login.
- Executed shell scripts
  - The shell can be programmed with specific shell languages (e.g., Bash)
  - Sequences of executable commands can be written with control structures and conditions in executable batch files, named “shell scripts”
  - Existing Linux Shell are:
    - BASH (Bourne Again SHell): the most diffused
    - CSH (C Shell)
    - TCSH (C Shell extended)
    - KSH (Shell Korn)



How to access to the SHELL?

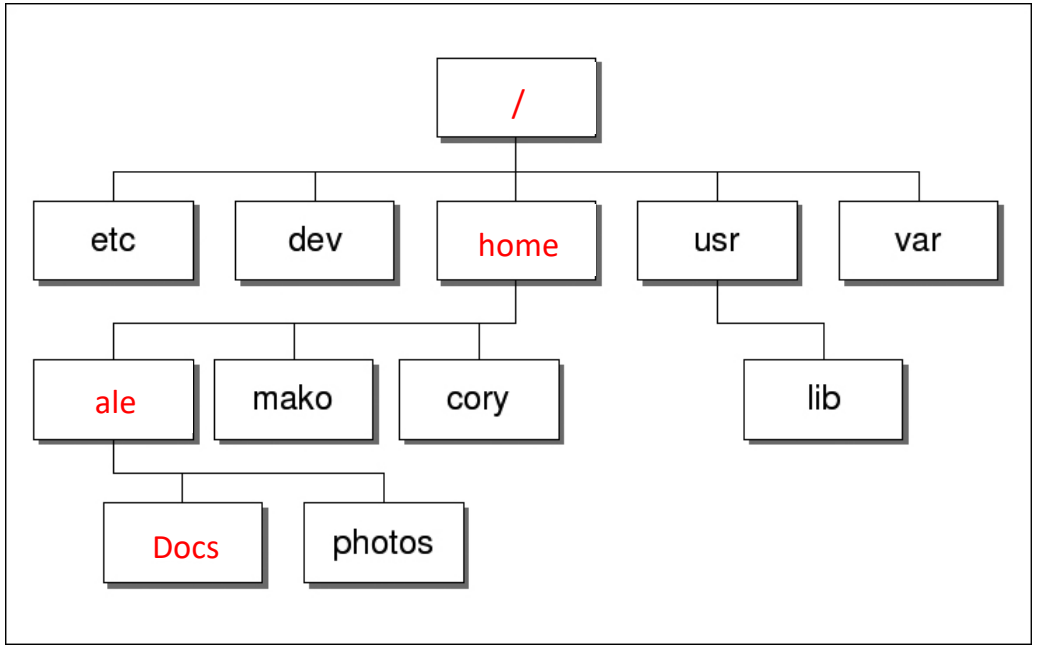
- CTRL+ALT+T
- Activity → write: “Terminal”

Linux Filesystem

- Linux OS filesystem, organization of the disk in folders

- Example of absolute path:

  /home/ale/Docs



Only a partial view as example

Linux Filesystem… *links to start*

- https://opensource.com/life/16/10/introduction-linux-filesystems
- https://www.linux.com/training-tutorials/linux-filesystem-explained
- https://wiki.archlinux.org/title/File_systems

(it focuses on the physical coding of files in the disk)

Linux (basic) SHELL commands

- cd ***name*** *(enter in folder called name)*
- ls *(list of files and folders)*
- mkdir ***dName*** *(make folder dName)*
- rm ***f*** *(remove the file f)*
- cat ***f*** *(concatenate the file f)*
- clear *(clear terminal)*

- man ***com*** *(manual of the command com)*
- cp ***file1 file2*** *(copy file1 in file2)*
- mv ***file1 file2*** *(move file1 in file2)*
- touch ***f*** *(create empty file f)*
- exit *(close the terminal)*
- pwd *(print working directory)*

Linux (basic) commands … *links*

Links to start:

- Cheatsheet of the main Linux commands

  https://www.geeksforgeeks.org/linux-unix/linux-commands-cheat-sheet
- Complete list of Linux command

  https://ss64.com/bash/
- Bash Guide For Beginners

  https://tldp.org/LDP/Bash-Beginners-Guide/html/index.html

  More complex, it focuses mainly on how to program the Shell by using the Bash language

Development environment for C++

- Linux distribution
  - Linux Ubuntu and similar
    - sudo apt-get install build-essential
  - Linux CentOS, RHEL, Fedora and similar
    - yum install gcc gcc-c++ kernel-devel make
    - yum groupinstall "Development Tools"
  - Linux Mint and similar
    - dnf groupinstall "Development Tools"





MS Windows

- Windows Linux Subsystem, and then Linux-like
  - https://docs.microsoft.com/en-us/windows/wsl/install
- MinGW
  - http://www.mingw.org
  - https://code.visualstudio.com/docs/cpp/config-mingw
- Cygwin - http://www.cygwin.com



MacOS

- Installing “Xcode” via App Store



Software program development

- Writing one or more text files, called source code of the program
- The source code is written in a specific programming language (e.g., C, C++, Java)
- The source code is printable, readable, comprehensive for humans
- “Editor ” is the tool to write and modify the source code
- Under Linux, e.g., Text Editor, emacs, vi, are editor for text (and source code)

Typically, in C++ the extension of a file containing source code of a program is “.cc”, “.cpp” (or “.h” or “.hpp”)

E.g., “test.cc”, “test.cpp”

Vi - Quick tutorial

- Open a file
  - vi <filename>
- Operation modes
  - Command mode
  - Insert mode
- Switch between mode
  - Command mode → Insert mode with “i”
  - Insert → command mode with “ESC”
- Getting out (in command mode)
  - “:q!” for exit without saving of changes
  - “:w” saving without exit
  - “:wq” for exit with saving of changes

Other editors:
- emacs
- nano
- gedit

https://ryanstutorials.net/linuxtutorial/vi.php
https://www.tutorialspoint.com/unix/unix-vi-editor.htm
https://www.cs.colostate.edu/helpdocs/vi.html

IDE

- Several Integrated Development Environments (IDEs) exist to support the developers in writing the code, making it “easy” and quick
  - E.g., Eclipse, Visual Studio Code, NetBeans, SonarLint, JetBrains, ….
- An IDE provides a suite of tools and features that aid development: code editor, easy compilation setup, a debugging tool, automatic execution, etc.
- Some IDEs are specific for a give programming language; others are more generic and can be configured to work with different languages
  - In this last case, a pre-configuration step is needed to setup the environment before being able to work with the desired language, e.g., setting the compiler to be used, the text editor

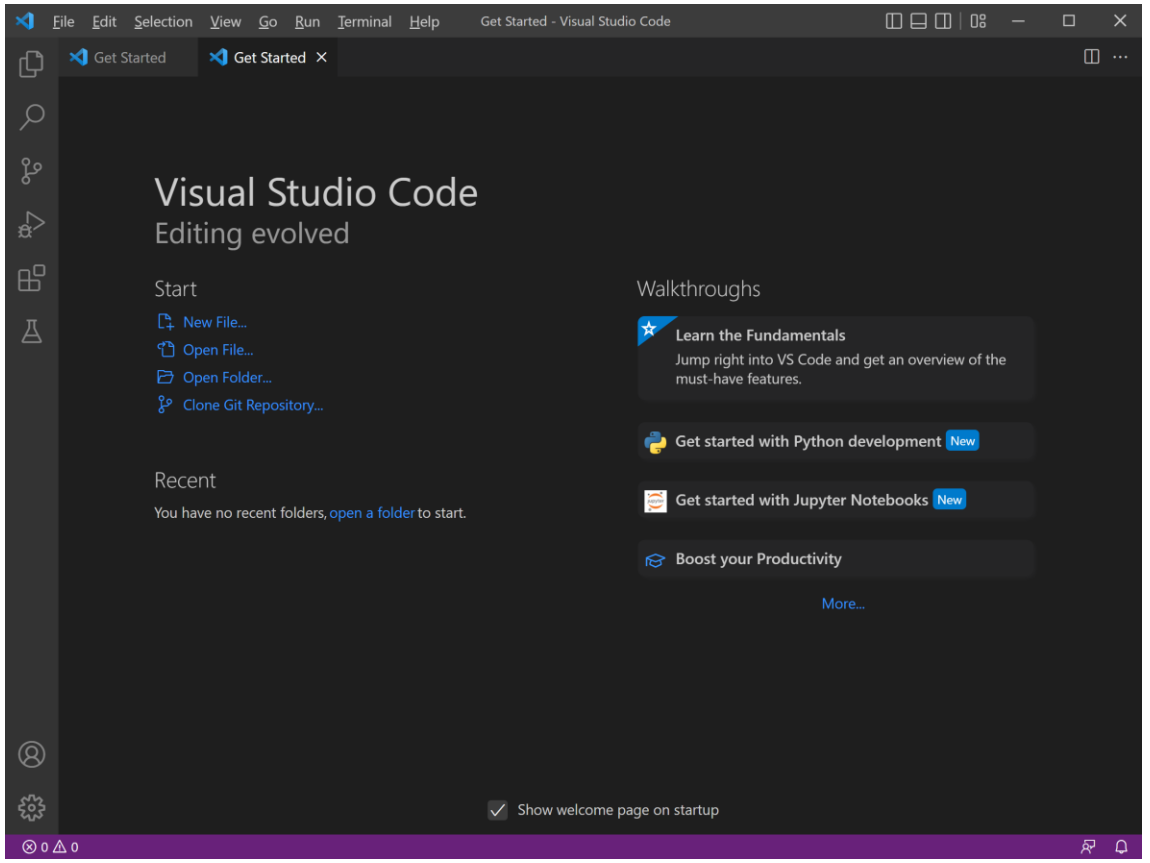
IDE

- Several Integrated Development Environments (IDEs) exist to support the developers in writing the code, making it “easy” and quick
  - E.g., ~~Eclipse~~, Visual Studio Code, ~~NetBeans~~, ~~SonarLint~~, ~~JetBrains~~, ….
- An IDE provides a suite of tools and features that aid development: code editor, easy compilation setup, a debugging tool, automatic execution, etc.
- Some IDEs are specific for a give programming language, others are more generic and can be configured to work with different languages
  - In this last case, a pre-configuration step is needed to setup the environment before being able to work with the desired language, e.g., setting the compiler to be used, the text editor



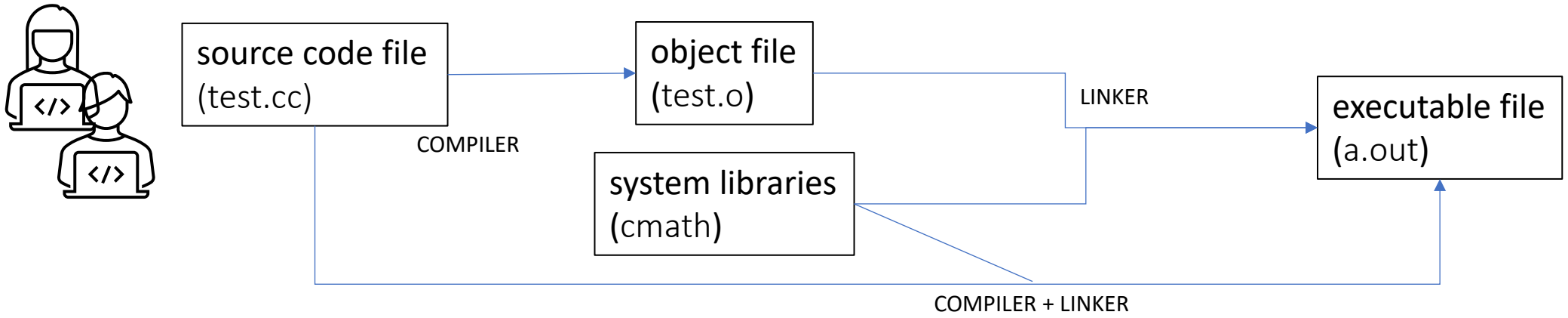
Visual Studio Code

- Platform:
  - Windows OS
  - WSL already installed
  - g++ compiler already installed in WSL



https://code.visualstudio.com

Software program development



- Source code file converted into an object file from the compiler
  - E.g., g++ -c test.cc
  - Imp: object file is not readable for humans
- Object file is linked to system libraries by the linker, thus generating the executable file (default a.out, optionally a specific name can be defined by using -o <name>)
  - E.g., g++ test.o
  - E.g., g++ test.o -o test
  - These files are unreadable for humans, but they are understandable and executable for computers
- File compiling and linking can be done also by means of a unique instruction
  - E.g., g++ test.cc

Software program development

- Compiler options
  - To specify a directory to search for (binary) libraries, use: -L:
    -L<dir>
  - To specify the actual library name, use: -l:
    -lfoo # (links libfoo.a or libfoo.so)
  - To specify a directory to search for include files (different from libraries!), use: -I:
    -I<dir>
  - To enable the generation of the source-level debug information, use: -g
  - To dis-enable the generation of the source-level debug information, use: -g0
  - To enable the generation of the warnings information, use: -Wall
  - To specify a name for the output file, use: -o:
    -o <name>
  - To show the commands to run and use verbose output, use: -v

E.g., g++ -g -Wall -I/data[...]/lib testing.cpp fileparameters.cpp main.cpp -o test

Empty program

- Open a Text Editor and create a file named example.cpp
- g++ example.cpp (to compile the program)
- ./a.out (to execute the program)

Example program:

```
int main()
{
   return 0;
}
```

Note:
Alt Gr + 7 → {
Alt Gr + 0 → }

… it’s your turn…

1. Switch on the PC, select Linux,

2. Try the first exercise (next slide)

3. Open a Browser, load the Moodle and looks for the exercises

E.g,

[LAB] Laboratory introduction and exercises on basic notions (Lecture 4)

[LAB] Basic exercises (Sept.16, 2026)

[LAB] Exercises on Basic notions and concepts - no streams (Sept.16, 2026)

[LAB][SRC] Exercises on Basic notions and concepts - no streams (Sept.16, 2026)

… it’s your turn…

1. Switch on the PC, select Linux,
2. Try the first exercise (next slide)
3. Open a Browser, load the Moodle and looks for the exercises

While completing the exercises, please use only the C++ concepts and instructions covered up to that point

E.g,

[LAB] Laboratory introduction and exercises on basic notions (Lecture 4)

[LAB] Basic exercises (Sept.16, 2026)

[LAB] Exercises on Basic notions and concepts - no streams (Sept.16, 2026)

[LAB][SRC] Exercises on Basic notions and concepts - no streams (Sept.16, 2026)

First exercise: Hello World!

Task #1

1. Log-in the Linux PC of the laboratory
2. Open a terminal
3. Open a Text Editor for creating a file “example.cpp”
4. Write the simple C++ HelloWord program
5. Compile it with the command *g++ example.cpp*
6. Run it with the command *./a.out*

Task #2

- Re-executed Task #1 with VisualStudio

Note:
Alt Gr + 7 → {
Alt Gr + 0 → }

First exercise: Hello World! … *solution*

```
// Header files required for using the output functions (we will come back to this aspect later in the course)
#include <iostream>
using namespace std;

// The main() function: where the execution of
int main() {

   // This statement prints "Hello World"
   cout << "Hello World";

   // This statement is only to conclude the program without reporting errors
   return 0;
}
```